DepGraph — Project Specification v0.1

> An open-source agent that reads your codebase and builds a living dependency graph across services, teams, and repositories — so blockers surface before they cost you a sprint.

---

1. Problem Statement

Engineering organisations above a certain size develop an invisible coordination tax. Teams block each other without knowing it. Status updates are written by humans from memory. Planning cycles end with misaligned timelines nobody caught. Informational calls exist purely because there is no shared source of truth.

The root cause is simple: **no system knows what depends on what**. Service catalogs require manual input and rot. Ticket trackers reflect intent, not reality. The only ground truth is the code — and nobody is reading it systematically.

DepGraph reads the code so engineers don’t have to track dependencies manually.

---

2. Vision

A self-hosted, open-source agent that:

- Connects to an organisation’s code store
- Crawls repositories and infers dependencies from source code
- Maps those dependencies to team ownership
- Surfaces the resulting graph as a queryable, living artifact
- Alerts on changes that carry cross-team risk

One-time setup. Always current. No human data entry.

---

3. Core Principles

Pull over declare. The system reads existing artifacts — source code, configuration, ownership files. It never asks engineers to maintain a separate registry.

Ground truth from code. Dependencies are inferred from what the code actually does, not from what teams report.

Self-hosted first. All data stays inside the organisation's environment. No external calls, no SaaS dependency for core functionality.

Contributor-friendly. Modular architecture. Each parser, connector, and output format is independently extensible.

---

4. Scope — Version 1

4.1 In Scope

Capability	Detail
Code store connectors	GitHub, AWS CodeCommit
Language parsers	Python, Java, JavaScript / TypeScript
Dependency types	Service-to-service calls, shared libraries, internal package imports
Ownership mapping	CODEOWNERS file, configurable team manifest
Graph output	JSON, DOT format (Graphviz-compatible)
CLI interface	Full feature access via command line
Change detection	Diff-based re-analysis on new commits
Alerting	Stdout + configurable webhook output

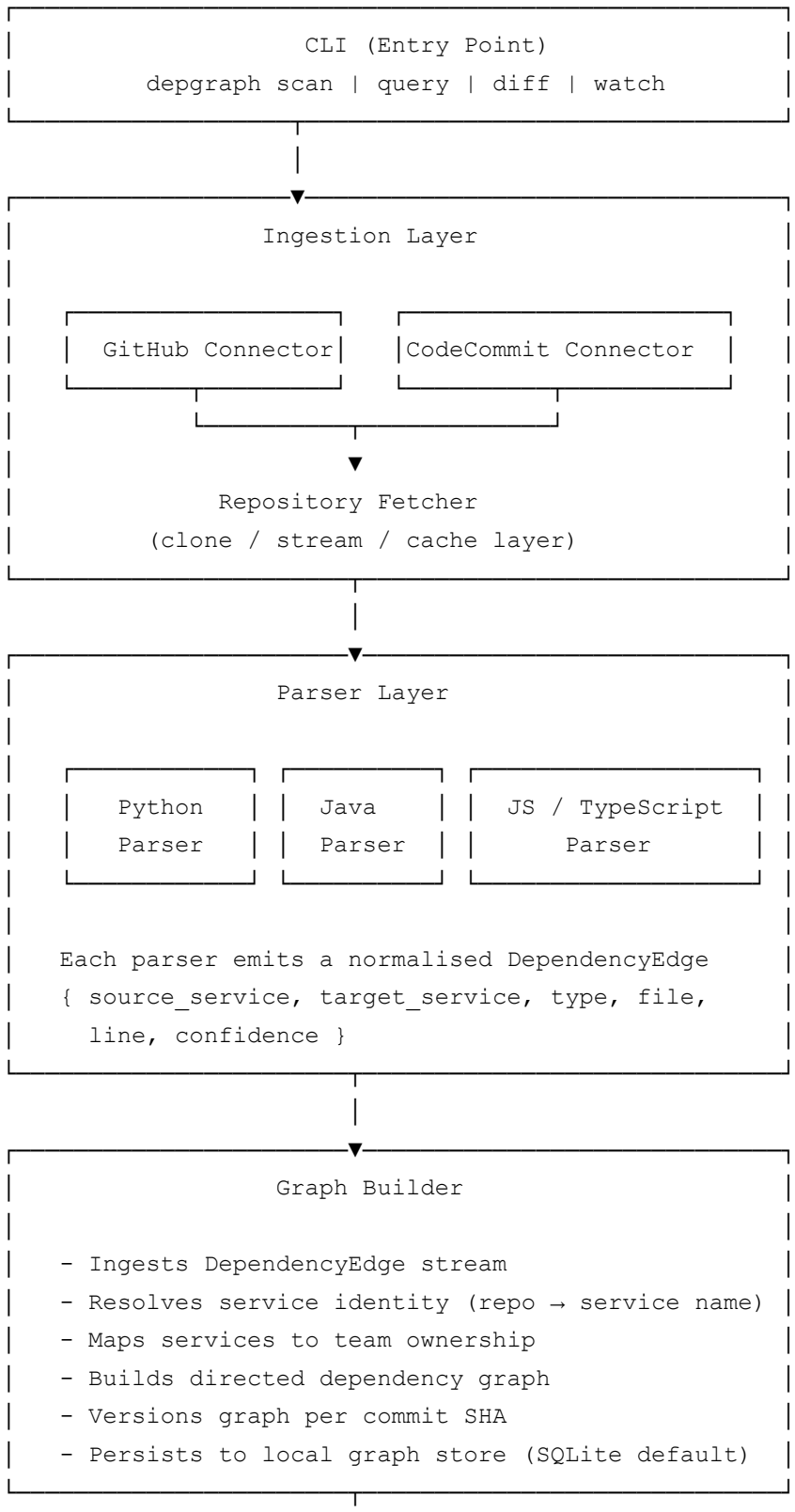
4.2 Explicitly Out of Scope for v1

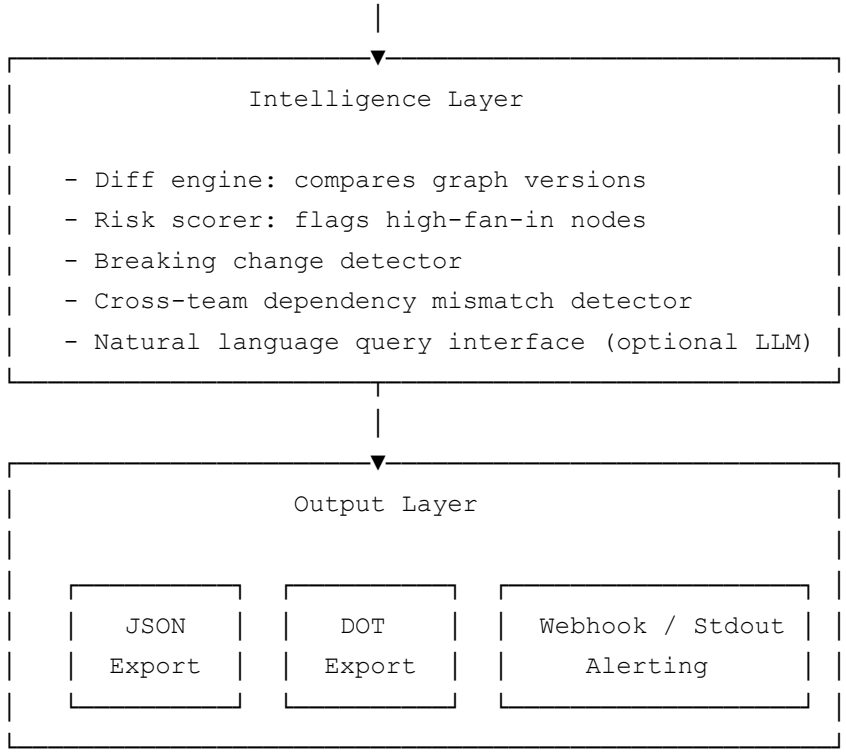
- Web UI / visual graph explorer
- Jira / Linear / GitHub Issues integration
- Inference from Slack or meeting data
- Multi-tenant or SaaS deployment mode
- Language support beyond Python, Java, JS/TS

- Runtime dependency tracing (observability/APM integration)

---

5. Architecture





---

6. Data Model

6.1 Service Node

```
{
  "id": "string",
  "name": "string",
  "repo": "string",
  "language": "python | java | javascript | typescript",
  "team": "string",
  "entry_points": ["string"],
  "last_analysed_sha": "string",
  "last_analysed_at": "ISO8601"
}
```

6.2 Dependency Edge

```
{
  "source_service": "string",
  "target_service": "string",
  "type": "http_call | import | shared_library | config_reference",
  "source_file": "string",
```

```
  "source_line": "integer",
  "confidence": "high | medium | low",
  "detected_at_sha": "string"
}
```

6.3 Graph Snapshot

```
{
  "snapshot_id": "string",
  "created_at": "ISO8601",
  "commit_shas": { "repo_name": "sha" },
  "nodes": ["ServiceNode"],
  "edges": ["DependencyEdge"]
}
```

---

7. Parser Specifications

7.1 Python Parser

Detects:

- `import` and `from X import Y` statements — resolves against known service list
- `requests.get/post/put/delete` calls — extracts URL patterns, matches against known service endpoints
- `boto3` client calls — flags AWS service dependencies
- `grpc` stubs — detects gRPC service calls

Tools: `ast` module (standard library) for AST-based parsing. No regex parsing of imports.

7.2 Java Parser

Detects:

- `import` statements — resolves against known internal package namespaces
- `RestTemplate`, `WebClient`, `HttpClient` call sites — extracts URL patterns
- `@FeignClient` annotations — direct service dependency declaration
- Maven / Gradle dependency declarations (`pom.xml`, `build.gradle`) — shared library dependencies

Tools: `javalang` library for AST parsing.

7.3 JavaScript / TypeScript Parser

Detects:

- `import` / `require` statements — resolves internal vs external packages
- `fetch` , `axios` , `got` call sites — extracts URL patterns
- `package.json` dependencies — shared library map
- TypeScript path aliases (`tsconfig.paths`) — resolved before import analysis

Tools: `@babel/parser` or `typescript` compiler API for AST parsing.

7.4 Confidence Scoring

Signal	Confidence
Explicit annotation (`@FeignClient` , service registry entry)	High
AST-resolved import matching known internal service	High
URL pattern matched against known service endpoint	Medium
Heuristic URL string match	Low

Low-confidence edges are included in the graph but flagged. Users can filter by confidence threshold.

---

8. Ownership Mapping

DepGraph resolves team ownership in the following priority order:

1. **CODEOWNERS file** — GitHub-standard, maps file path patterns to team handles
2. **depgraph.yaml manifest** — explicit service-to-team mapping, for repos without CODEOWNERS
3. **Fallback** — unowned; flagged in output

depgraph.yaml format

```
version: 1
services:
  - name: payments-service
    repo: org/payments
    team: payments-team
    entry_points:
      - src/main/java/com/org/payments/PaymentsApplication.java
  - name: auth-service
    repo: org/auth
    team: platform-team
```

---

9. CLI Interface

Commands

```
# Initialise config in current directory
depgraph init

# Run full scan across all configured repos
depgraph scan

# Scan a specific repo
depgraph scan --repo org/payments

# Query the graph
depgraph query --service payments-service --direction upstream
depgraph query --team platform-team
depgraph query --impact-of auth-service   # what breaks if this changes

# Show diff between two snapshots
depgraph diff --from <snapshot_id> --to <snapshot_id>
depgraph diff --last 2   # compare two most recent snapshots

# Watch mode: re-scan on new commits
depgraph watch --interval 30m

# Export graph
depgraph export --format json --output graph.json
depgraph export --format dot --output graph.dot

# Show alerts
depgraph alerts
depgraph alerts --severity high
```

Config file — `.depgraph/config.yaml`

```yaml
version: 1

connectors:
  github:
    org: your-org
    token_env: DEPGRAPH_GITHUB_TOKEN
    repos:
      - payments
      - auth
      - notification-service

  codecommit:
    region: us-east-1
    profile_env: DEPGRAPH_AWS_PROFILE
    repos:
      - arn:aws:codecommit:us-east-1:123456789:payments-legacy

parsers:
  languages:
    - python
    - java
    - typescript
  confidence_threshold: medium   # low | medium | high

ownership:
  strategy: codeowners           # codeowners | manifest | both
  manifest_path: .depgraph/depgraph.yaml

storage:
  backend: sqlite                # sqlite (default) | postgres
  path: .depgraph/graph.db

alerts:
  on_breaking_change: true
  on_new_cross_team_dependency: true
  webhook_url_env: DEPGRAPH_WEBHOOK_URL
```

---

10. Alert Definitions

Alert		Severity

	Trigger	
`BREAKING_CHANGE`	Public interface removed or signature changed in a service with known dependents	High
`NEW_CROSS_TEAM_DEPENDENCY`	A new dependency edge crosses team ownership boundary	Medium
`ORPHANED_SERVICE`	Service has no dependents and no dependencies after a scan	Low
`HIGH_FAN_IN`	Service gains dependents beyond configured threshold (default: 10)	Medium
`UNOWNED_SERVICE`	Service detected with no resolvable team owner	Low

---

11. Repository Structure

```
depgraph/
├── depgraph/
│   ├── cli/                    # CLI entry points and command handlers
│   ├── connectors/
│   │   ├── base.py
│   │   ├── github.py
│   │   └── codecommit.py
│   ├── parsers/
│   │   ├── base.py
│   │   ├── python_parser.py
│   │   ├── java_parser.py
│   │   └── js_ts_parser.py
│   ├── graph/
│   │   ├── builder.py
│   │   ├── models.py
│   │   └── store.py
│   ├── intelligence/
│   │   ├── diff.py
│   │   ├── risk.py
│   │   └── alerts.py
│   ├── ownership/
│   │   ├── codeowners.py
│   │   └── manifest.py
│   └── output/
│       ├── json_export.py
```

```
│       └── dot_export.py
├── tests/
│   ├── fixtures/               # Sample repos for parser testing
│   ├── test_parsers/
│   ├── test_graph/
│   └── test_connectors/
├── docs/
│   ├── SPEC.md                 # This document
│   ├── CONTRIBUTING.md
│   └── ARCHITECTURE.md
├── .depgraph/
│   └── config.yaml.example
├── pyproject.toml
├── README.md
└── LICENSE                     # Apache 2.0
```

---

12. Non-Functional Requirements

Requirement	Target
Scan time for 50-repo org	< 10 minutes on initial scan
Incremental scan (diff-based)	< 2 minutes
Memory footprint	< 512 MB for orgs up to 200 repos
Python version support	3.10+
External network requirement	None (beyond code store access)
Auth credential storage	Environment variables only — never written to disk

---

13. Open Questions (Pre-Implementation)

These are decisions to resolve before or during implementation:

1. **Graph storage at scale** — SQLite is sufficient for v1 but may not hold for orgs with 500+ repos and years of history. Should Postgres be the recommended default from day one?
2. **URL-to-service resolution** — Matching a raw URL string like

`https://payments.internal/charge` to a service node requires a URL registry. Where does this live — in `depgraph.yaml`, inferred from Kubernetes ingress definitions, or both?

3. **Monorepo handling** — When multiple services live in a single repo, how does service boundary detection work? Heuristics (one service per top-level directory) or explicit config?
4. **LLM integration** — The natural language query interface (`"what teams are affected if auth-service goes down?"`) is optional in v1. Should it be a first-class feature with optional enablement, or a v2 addition?
5. **License** — Apache 2.0 is proposed. Any objection before the repo is created?

---

14. Milestones

Milestone	Deliverables
M0 — Foundation	Repo setup, CLI scaffold, config parsing, GitHub connector (read-only)
M1 — First Graph	Python parser complete, graph builder, SQLite store, JSON export
M2 — Full Parser Coverage	Java and JS/TS parsers, CODEOWNERS ownership mapping
M3 — Intelligence	Diff engine, alert system, CodeCommit connector
M4 — Polish	DOT export, depgraph.yaml manifest support, full test coverage, contributor docs

---

Spec version 0.1 — subject to revision based on community input.